Inventory & Order Management API

Enhancement Report

By Alharith Alkindi

Abstract

This report presents the design, development, and testing of an Inventory and Order Management REST API developed as part of the Codeline Graduate Technical Assignment. The system was built using Java 17, Spring Boot 3.5.15, Spring Data JPA, and Oracle Database 19c, providing full backend functionality for a small online store. The API covers product and category management, customer registration, and a complete order lifecycle spanning draft creation through confirmation, shipping, delivery, and cancellation.

The central technical challenge was implementing atomic stock management — ensuring that inventory is deducted exclusively at the moment of order confirmation and fully restored upon cancellation, with concurrent orders correctly prevented from overselling. This was achieved through a two-pass confirmation strategy encapsulated in a @Transactional service method.

The project adheres to a strict layered architecture (Controller → Service → Repository), applies database normalization up to Third Normal Form (3NF), and was validated through 20 Postman test cases covering both standard operational flows and business rule violations. All five evaluation competencies — System Design, OOP Analysis, Database Structure, API Design, and Testing — were systematically addressed throughout the development process.

1. Introduction

Modern e-commerce systems rely on backend APIs to manage the flow of products, inventory, and customer orders reliably and consistently. A core challenge in such systems is ensuring that stock levels remain accurate at all times — particularly when multiple customers may be competing for the same limited inventory simultaneously.

This report documents the development of the Inventory and Order Management API, a graduate-level assignment designed to simulate the type of backend problem a junior engineer would encounter within a real product team. The assignment required building a Spring Boot REST API backed by a relational SQL database, implementing a set of business rules that extend beyond simple CRUD operations, and validating those rules through systematic API testing.

1.1 Objectives

The primary objectives of this assignment were as follows:

- Design a normalized relational schema derived from a written business scenario.
- Implement a layered Spring Boot application with proper separation of concerns.
- Enforce critical business rules — particularly the stock-on-confirmation rule.
- Validate all functionality through systematic API testing using Postman.

1.2 Scope

The system manages five core entities — Categories, Products, Customers, Orders, and Order Items — extended with two supplementary tables: CUSTOMERS_PHONE, for storing multiple phone numbers per customer in compliance with First Normal Form (1NF), and ORDER_STATUS_HISTORY, for maintaining a complete audit trail of every order status transition.

1.3 Technology Stack

Component	Technology
Language	Java 17
Framework	Spring Boot 3.5.15
ORM	Spring Data JPA / Hibernate 6.6
Database	Oracle Database 19c (orclpdb)
API Testing	Postman
Build Tool	Maven
IDE	IntelliJ IDEA

1.4 Report Structure

This report is organized as follows:

- Section 2 — Project Overview: General description of the project scope and development environment.
- Section 3 — Challenges Faced: Technical obstacles encountered during development.
- Section 4 — Issues and Bugs Identified: Documented defects and their resolutions.
- Section 5 — Technical Decisions: Key architectural and design decisions with justifications.
- Section 6 — Postman Test Cases: All test cases with screenshots and expected results.
- Section 7 — Suggestions for Improvement: Recommendations for future enhancements.
- Section 8 — Lessons Learned: Key takeaways from the development experience.
- Section 9 — Conclusion: Summary of outcomes and project achievements.

2. Project Overview

This report documents the individual development experience of the Inventory and Order Management API — a two-day take-home assignment for fresh Java and Spring Boot graduates. The project required building a fully functional backend REST API for a small online store managing products, categories, customers, and orders.

The system was built using Java 17, Spring Boot 3.5.15, Spring Data JPA, and Oracle Database 19c (orclpdb). The API was validated using Postman with a suite of 15 test cases covering standard operational flows and business rule violations.

3. Challenges Faced

3.1 Oracle Database Configuration

Establishing a connection between Spring Boot and Oracle Database presented the first unexpected challenge. Attempting to create a user from the root Container Database (CDB) produced an ORA-65096 error. Resolution required switching to the Pluggable Database (PDB) first by executing ALTER SESSION SET CONTAINER = orclpdb prior to user creation.

3.2 Stock Confirmation Logic

The most intellectually demanding aspect of the project was the correct implementation of the stock reservation rule — inventory must not be deducted when a product is added to a DRAFT order; deduction occurs exclusively upon order CONFIRMATION. This requirement was fulfilled using a two-pass approach: all order items are validated in a first pass, followed by atomic stock deduction in a second pass, all wrapped in a @Transactional service method.

3.3 Missing Maven Dependencies

Upon generating the project from start.spring.io, essential dependencies — including Spring Data JPA, the Oracle JDBC Driver, Bean Validation, and Lombok — were absent, resulting in more than 15 compilation errors immediately on project import. These dependencies were added manually to the pom.xml configuration file and Maven was reloaded to resolve the issue.

3.4 Circular Reference in JSON Serialization

Bidirectional JPA relationships between entities caused infinite recursion in JSON responses. This was resolved by applying the @JsonManagedReference annotation to all OneToMany relationships and the @JsonBackReference annotation to their corresponding ManyToOne counterparts across all affected entity classes.

3.5 Mid-Development Schema Changes

The database schema underwent significant revision during development, including splitting the customer name field into first_name and last_name, introducing the CUSTOMERS_PHONE table, and adding the ORDER_STATUS_HISTORY table. These changes necessitated simultaneous updates to all affected entities, services, controllers, and repositories.

4. Issues and Bugs Identified

The following table documents all defects identified during development, along with their symptoms and resolutions.

#	Bug	Symptom	Resolution
1	ORA-65096 on User Creation	Cannot create DB user in SQL Plus	Switch to PDB container first using ALTER SESSION SET CONTAINER = orclpdb
2	Missing Maven Dependencies	15+ compilation errors in IntelliJ	Added JPA, ojdbc11, Validation, and Lombok to pom.xml
3	Circular JSON Reference	Infinite nested API response	Applied @JsonManagedReference / @JsonBackReference across all entity relationships
4	409 Conflict via /status Endpoint	Invalid status transition error	Introduced a dedicated POST /cancel endpoint
5	500 Error on /history Endpoint	Internal server error	Added findByOrderOrdId() method to the repository
6	Lazy Loading in cancelOrder()	500 when cancelling a confirmed order	Applied FetchType.EAGER and @Transactional annotations

5. Technical Decisions

5.1 Atomic Stock Deduction on Confirmation

Stock is deducted exclusively at the moment of order confirmation. The implementation employs a two-pass approach: all items are validated in the first pass to confirm sufficient stock availability, followed by atomic deduction in the second pass. The @Transactional annotation ensures a full rollback in the event of any failure during the process, thereby preventing overselling under concurrent order scenarios.

5.2 Price Locking at Confirmation

The unit_price column in the ORDER_ITEMS table captures the product price at the time of order confirmation. Subsequent changes to the product price do not affect previously confirmed orders, ensuring pricing consistency and auditability.

5.3 CUSTOMERS_PHONE Table

Customer phone numbers are stored in a dedicated CUSTOMERS_PHONE table rather than as a multi-valued column in the CUSTOMERS table. This design complies with First Normal Form (1NF) and supports scenarios where a customer possesses multiple phone numbers.

5.4 ORDER_STATUS_HISTORY Table

Every order status transition is recorded in the ORDER_STATUS_HISTORY table, capturing the old_status, new_status, and changed_at timestamp. This provides a complete and immutable audit trail of order lifecycle events.

5.5 Centralized Exception Handling

A single class annotated with @RestControllerAdvice serves as the global exception handler for the entire application. It maps specific exception types to appropriate HTTP status codes: 404 for resource not found, 409 for business rule violations, and 500 for unexpected system errors. This approach eliminates repetitive try-catch blocks from all controller classes.

5.6 Layered Architecture

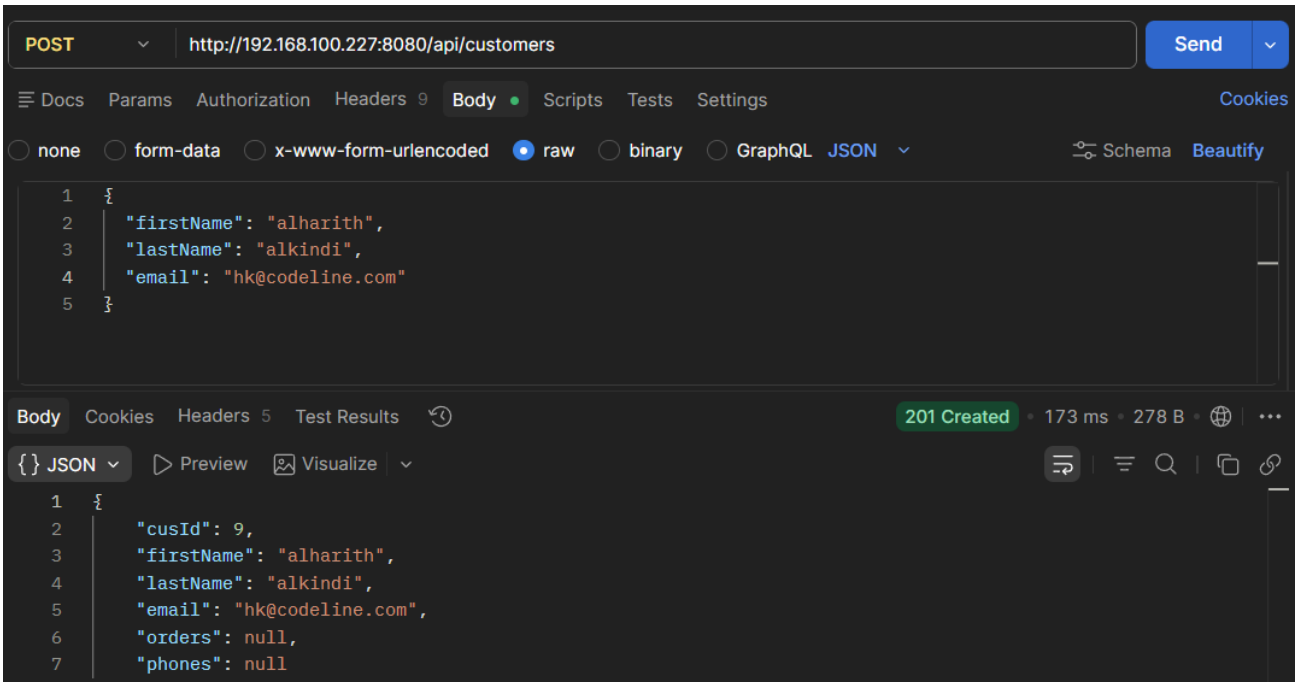
The application enforces a strict three-layer architecture: Controller → Service → Repository. All business logic and rules are encapsulated exclusively within the Service layer, ensuring a clean separation of concerns and facilitating maintainability and testability.

6. Postman Test Cases

The following test cases were executed using Postman to validate both standard operational flows and business rule enforcement. Each test specifies the HTTP method, endpoint URL, request body (where applicable), expected response, and a screenshot of the actual result.

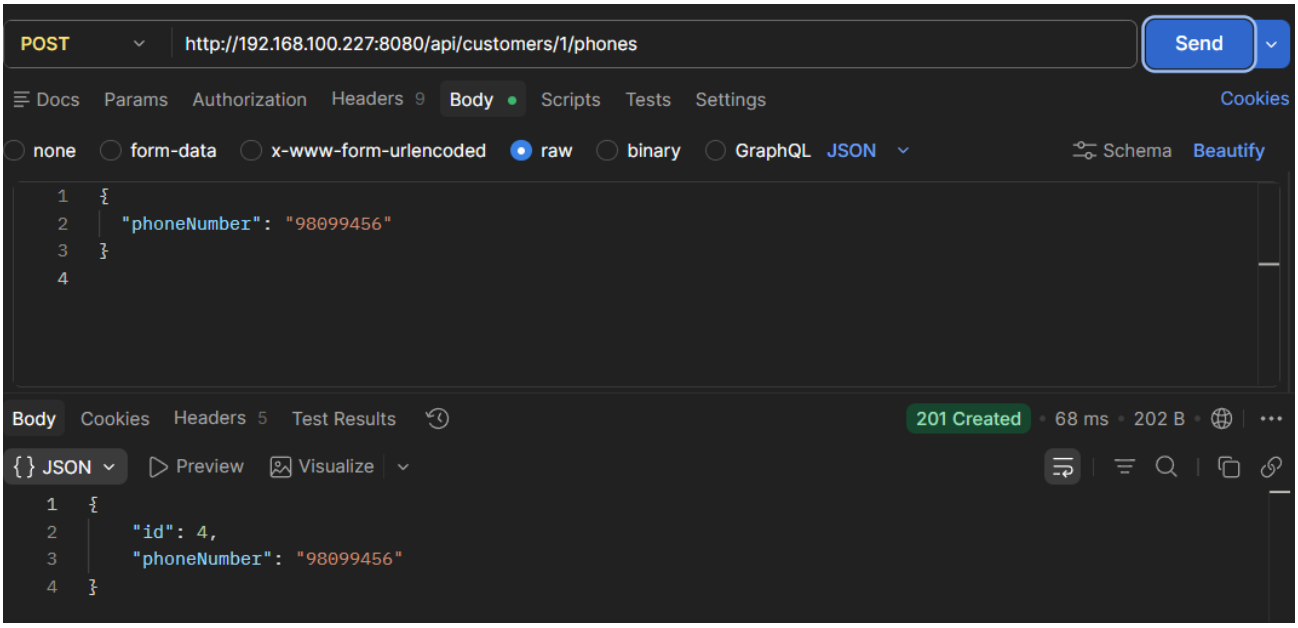
Test 1 — Create Customer	
Method	POST
URL	http://192.168.100.227:8080/api/customers
Request Body	{ "firstName": "alharith", "lastName": "alkindi", "email": "alharith@codeline.com" }
Expected Response	201 Created

Screenshot:



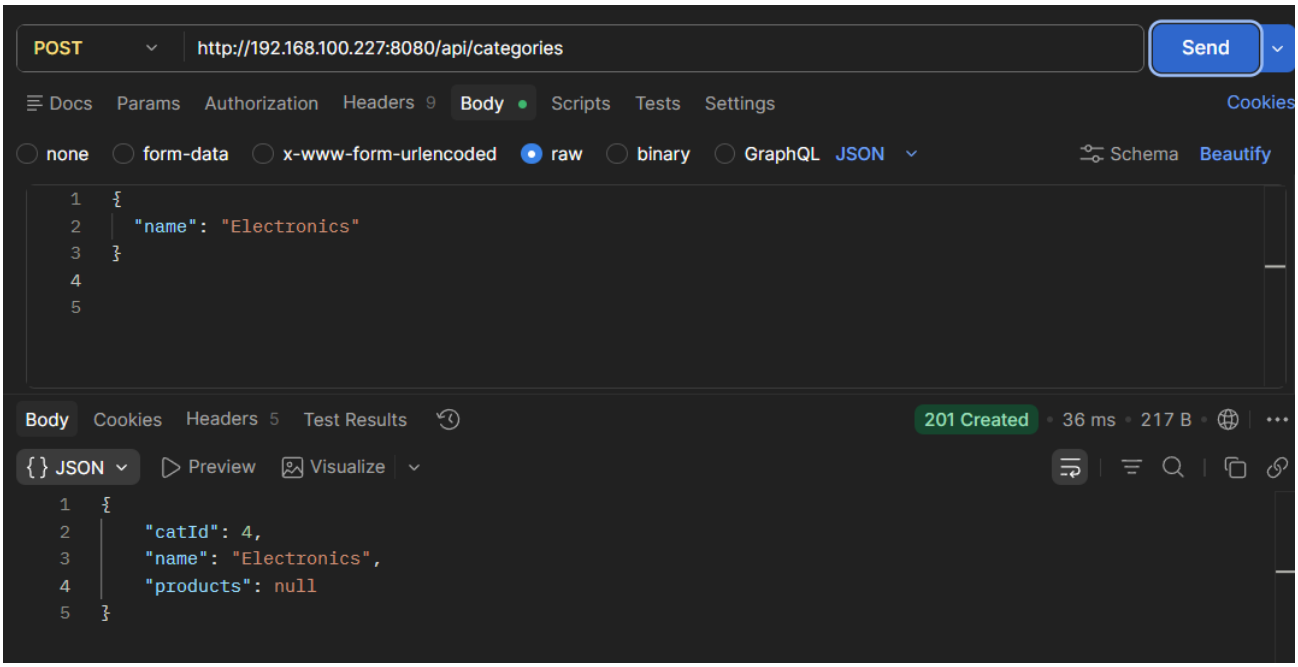
Test 2 — Add Phone Number to Customer	
Method	POST
URL	http://192.168.100.227:8080/api/customers/1/phones
Request Body	{ "phoneNumber": "98099456" }
Expected Response	201 Created

Screenshot:



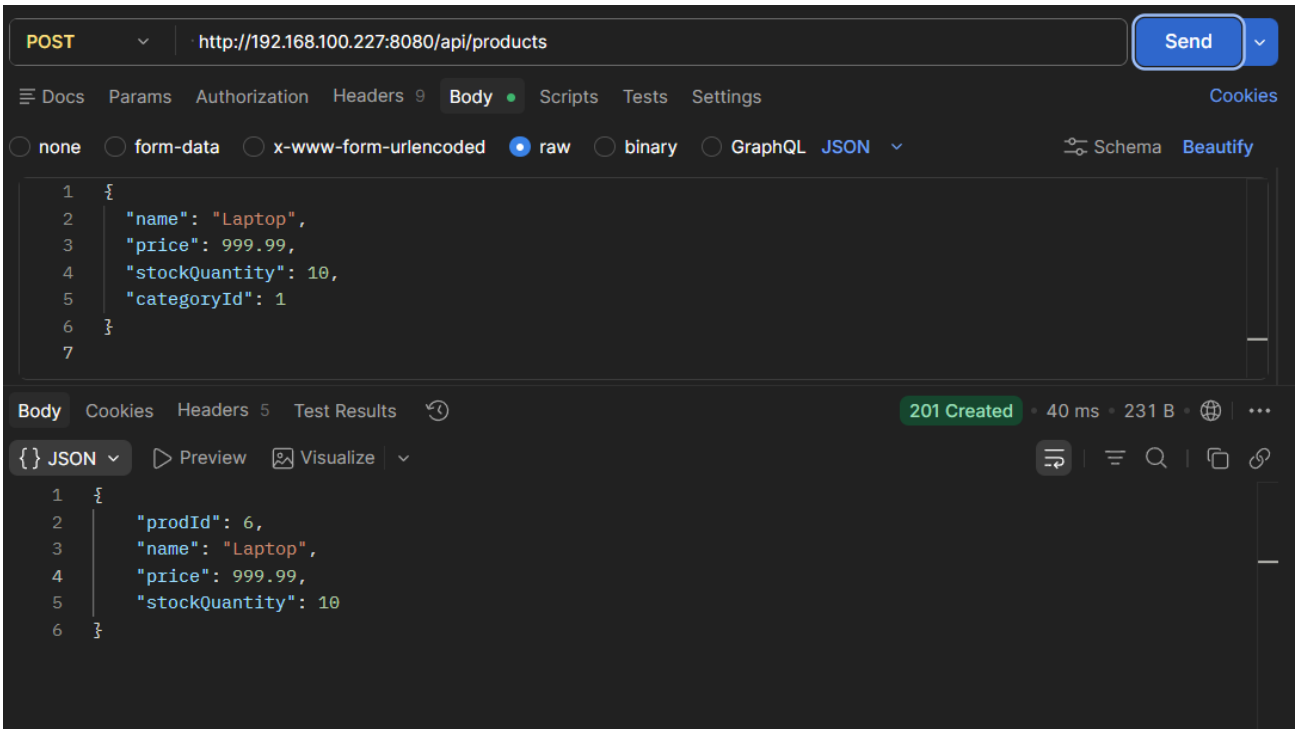
Test 3 — Create Category	
Method	POST
URL	http://192.168.100.227:8080/api/categories
Request Body	{ "name": "Electronics" }
Expected Response	201 Created

Screenshot:



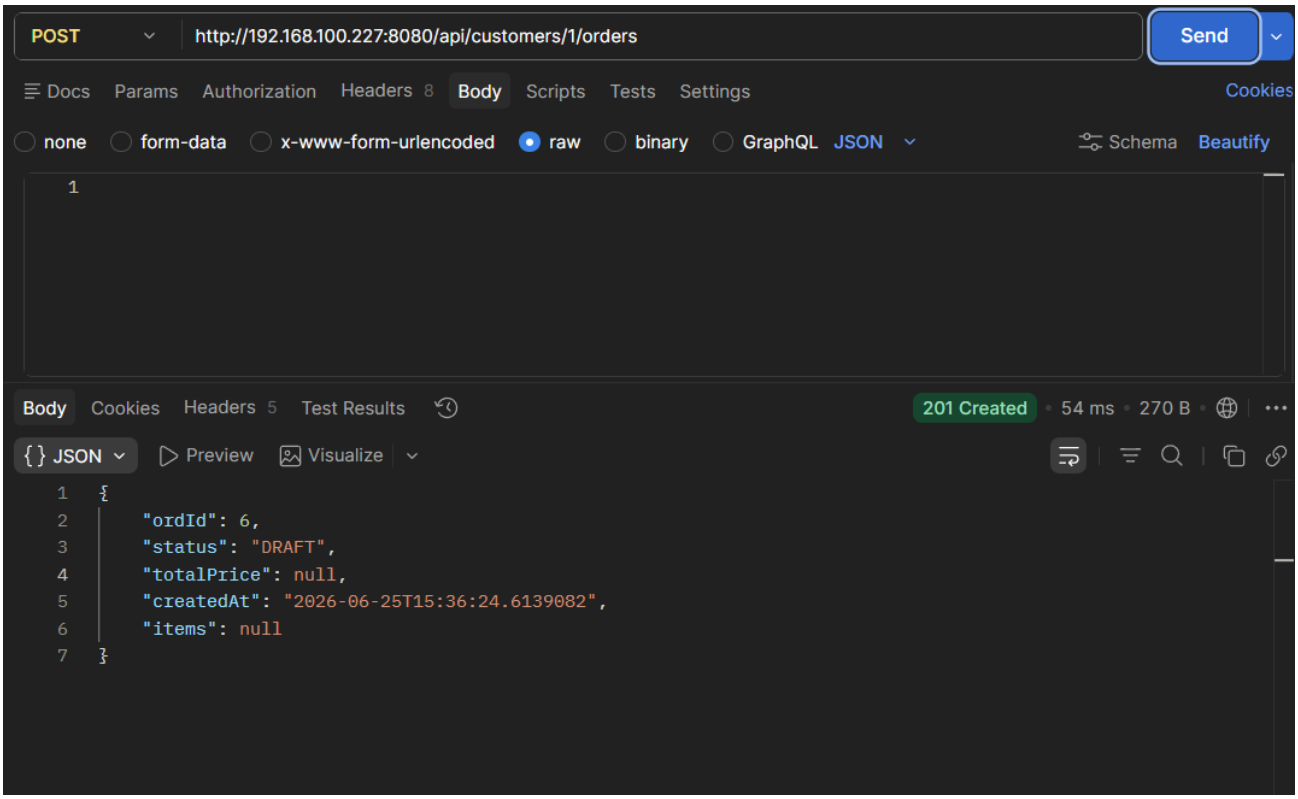
Test 4 — Create Product	
Method	POST
URL	http://192.168.100.227:8080/api/products
Request Body	{ "name": "Laptop", "price": 999.99, "stockQuantity": 10, "categoryId": 1 }
Expected Response	201 Created

Screenshot:



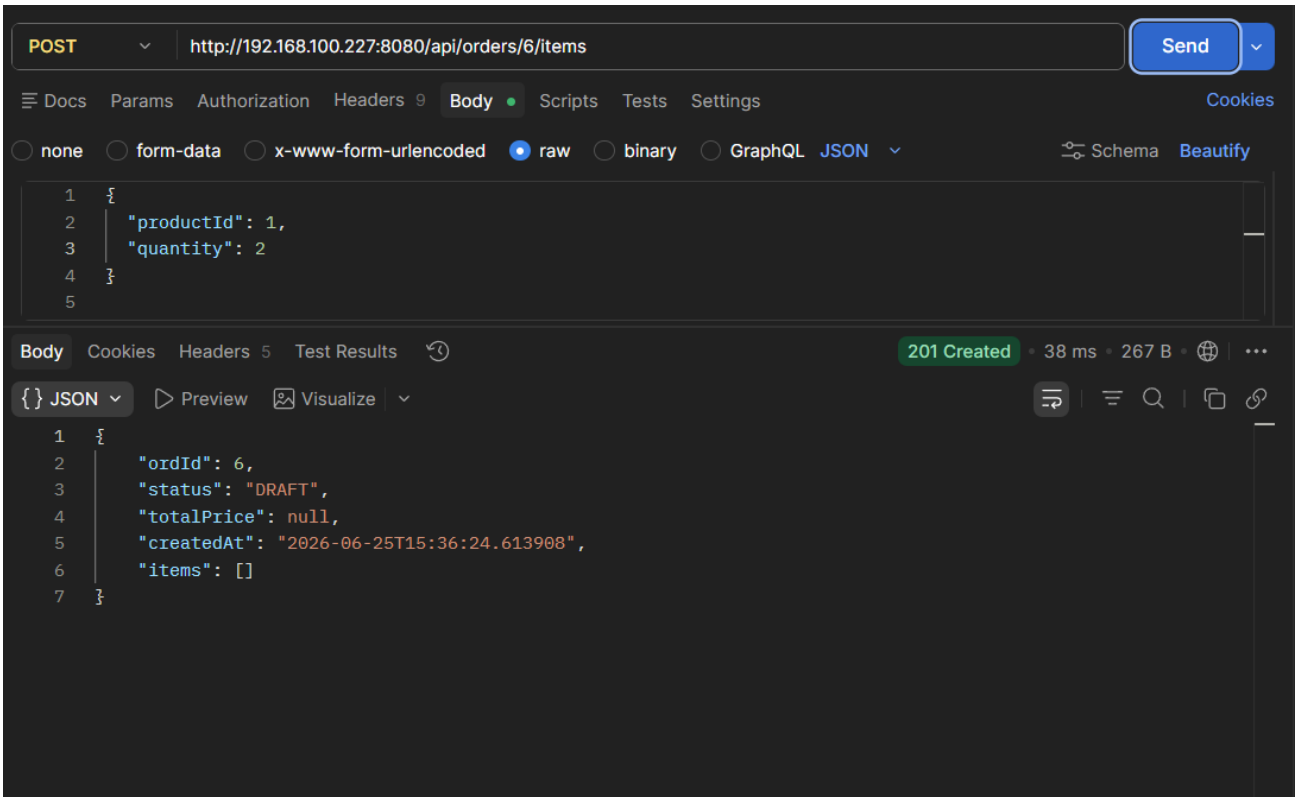
Test 5 — Create Draft Order	
Method	POST
URL	http://192.168.100.227:8080/api/customers/1/orders
Request Body	{}
Expected Response	201 Created — Response body includes: status: "DRAFT"

Screenshot:



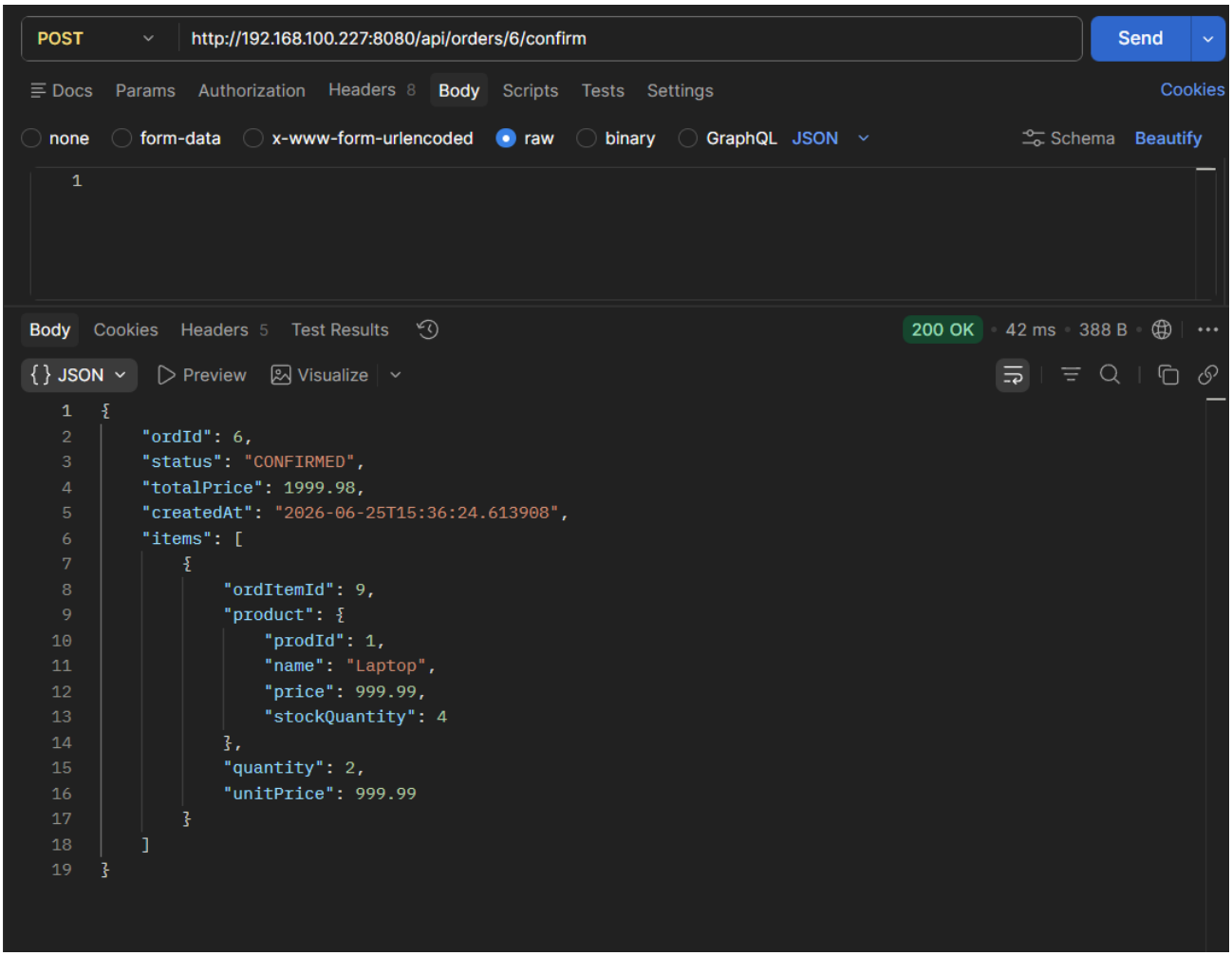
Test 6 — Add Item to Order	
Method	POST
URL	http://192.168.100.227:8080/api/orders/6/items
Request Body	{ "productId": 1, "quantity": 2 }
Expected Response	201 Created

Screenshot:



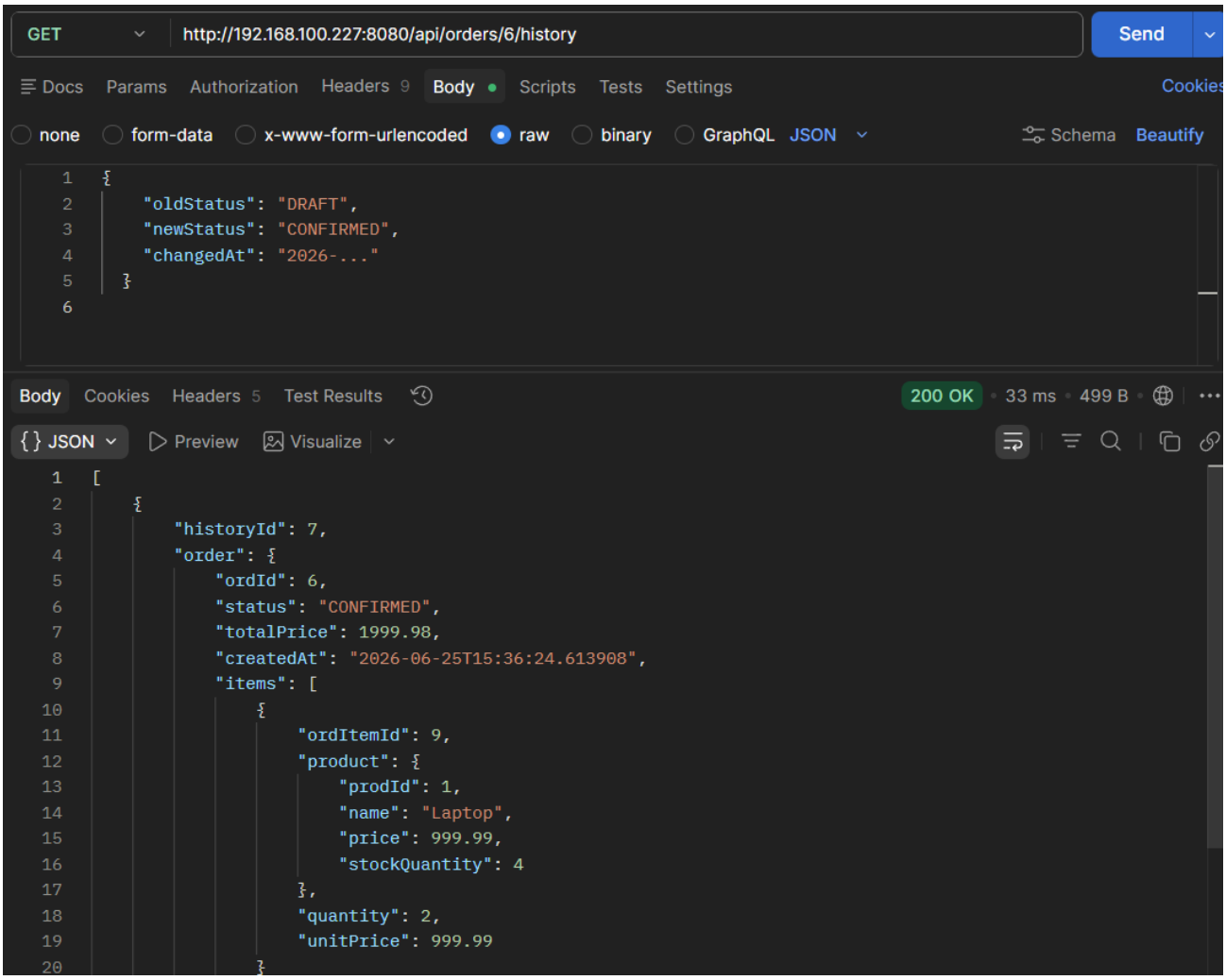
Test 7 — Confirm Order	
Method	POST
URL	http://192.168.100.227:8080/api/orders/1/confirm
Expected Response	200 OK — Response body includes: status: "CONFIRMED"

Screenshot:



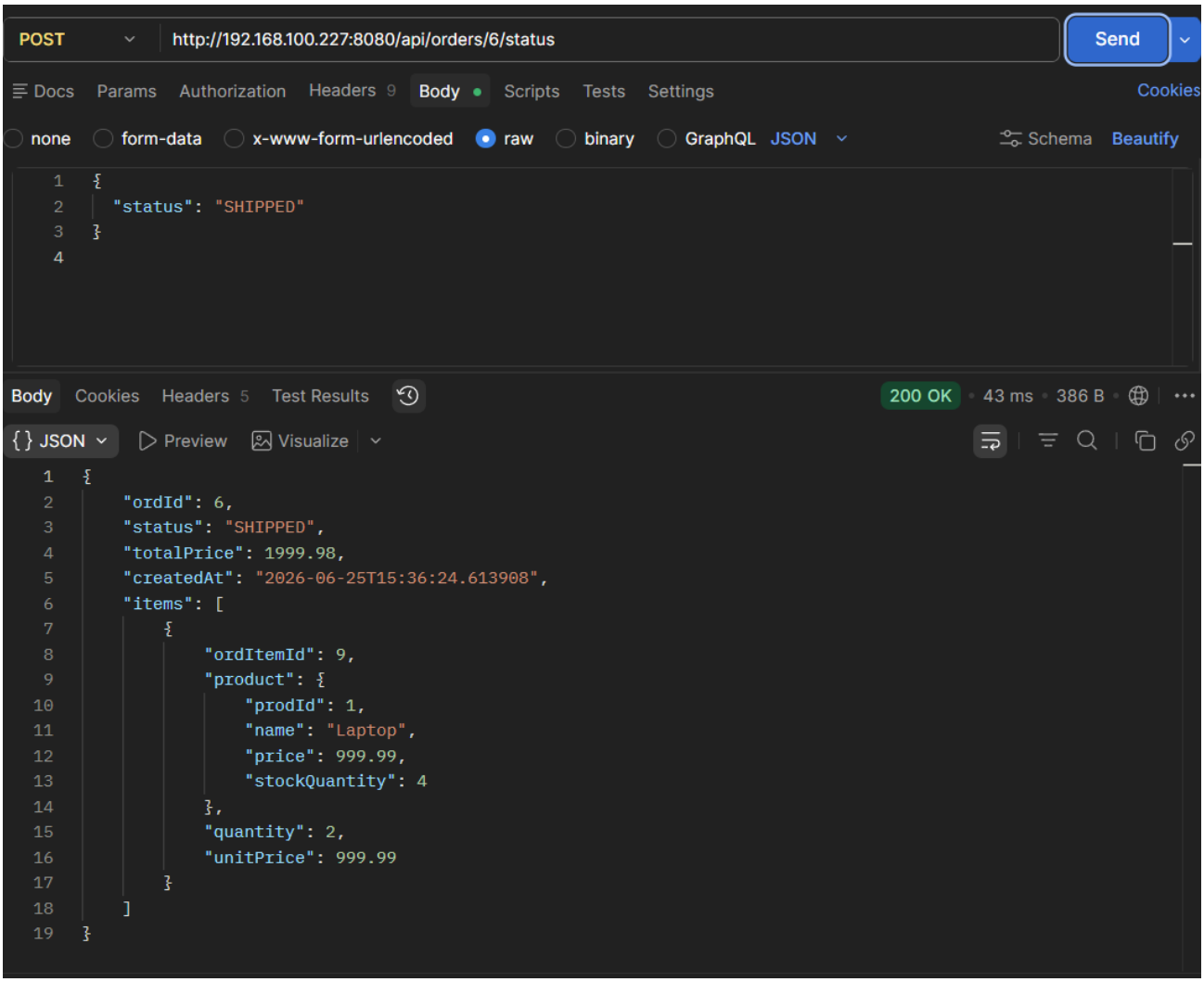
Test 8 — Retrieve Status History (DRAFT → CONFIRMED)	
Method	GET
URL	http://192.168.100.227:8080/api/orders/1/history
Expected Response	200 OK — Response includes: { "oldStatus": "DRAFT", "newStatus": "CONFIRMED", "changedAt": "2026-..." }

Screenshot:



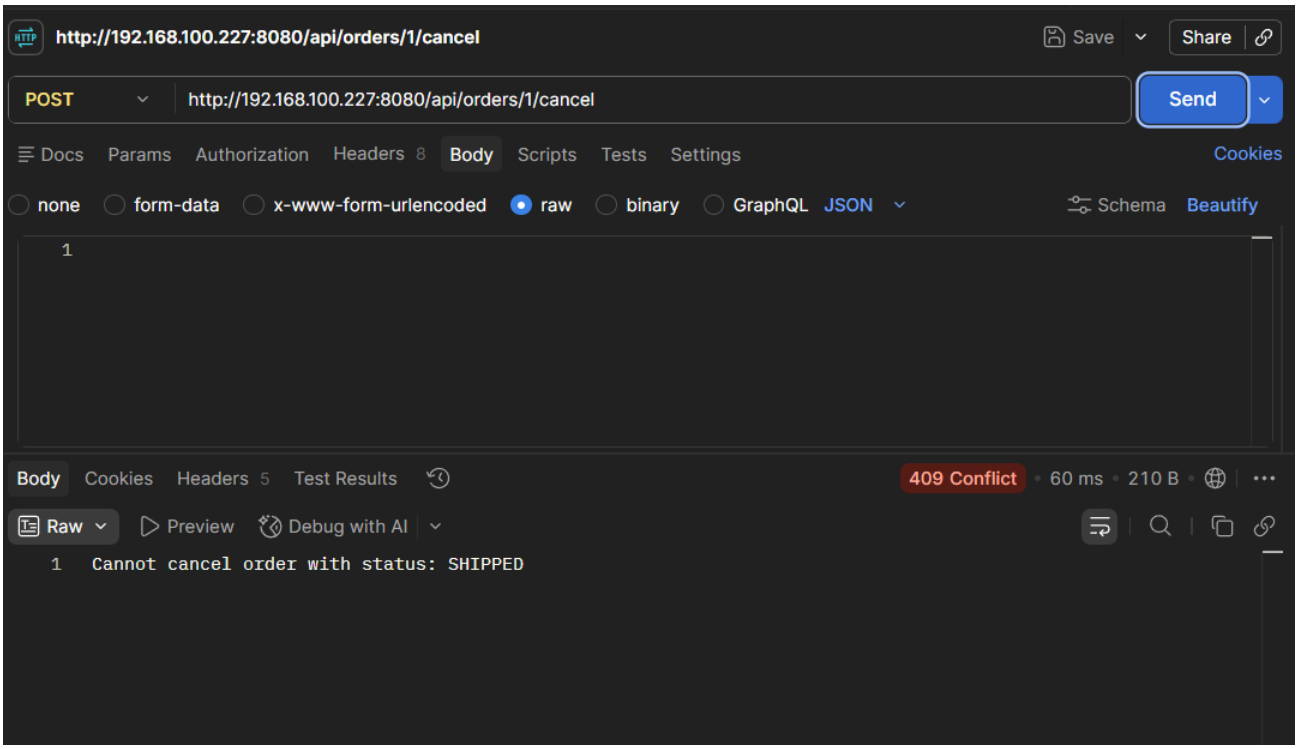
Test 9 — Ship Order	
Method	POST
URL	http://192.168.100.227:8080/api/orders/6/status
Request Body	{ "status": "SHIPPED" }
Expected Response	200 OK — Status history now contains 2 records

Screenshot:



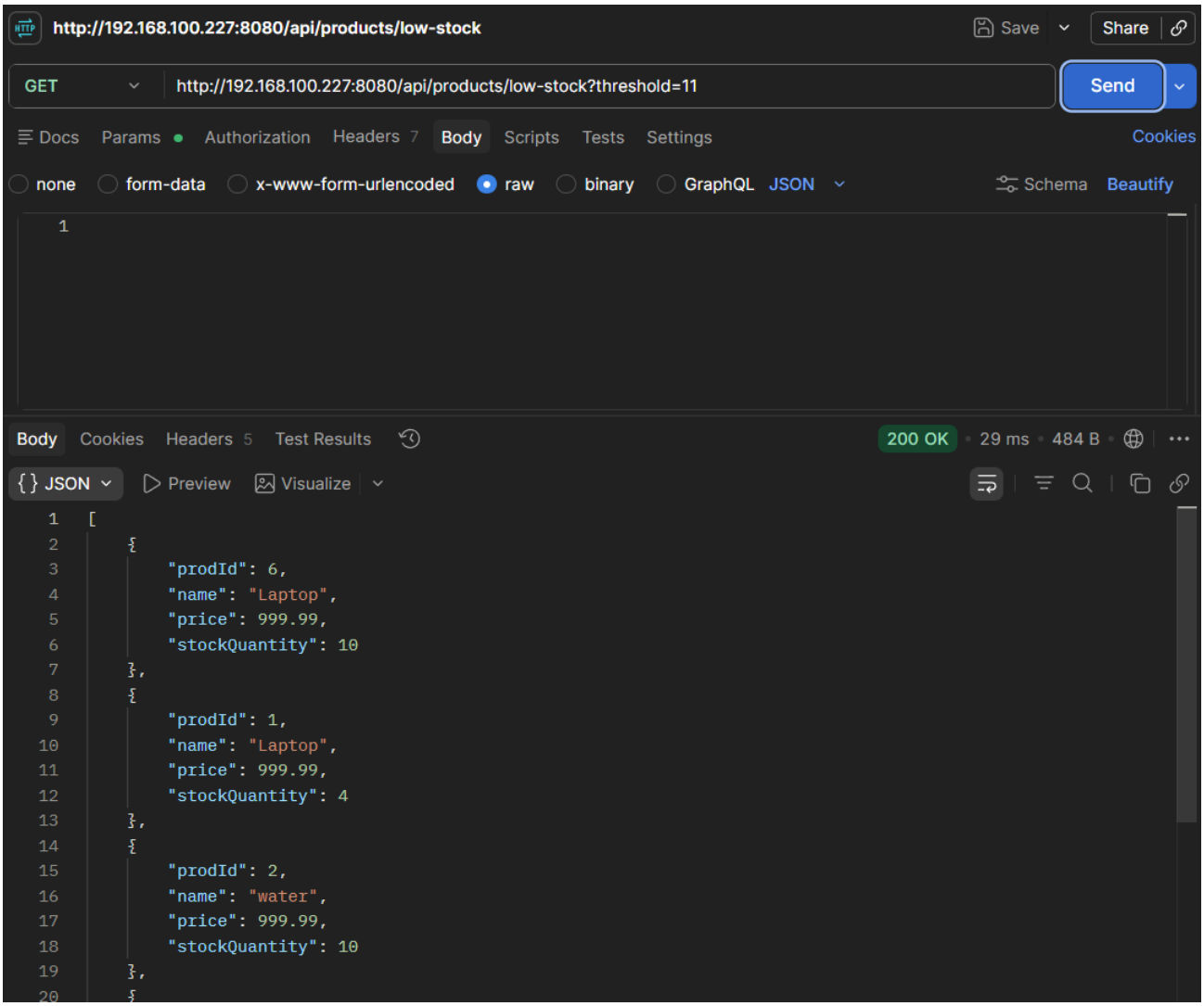
Test 10 — Attempt to Cancel a Shipped Order	
Method	POST
URL	http://192.168.100.227:8080/api/orders/1/cancel
Expected Response	409 Conflict — Cannot cancel an order with status SHIPPED

Screenshot:



Test 11 — Low Stock Report	
Method	GET
URL	http://192.168.100.227:8080/api/products/low-stock?threshold=11
Expected Response	200 OK — Returns Laptop with stockQuantity: 8

Screenshot:


http://192.168.100.227:8080/api/products/low-stock
Save
Share
GET
http://192.168.100.227:8080/api/products/low-stock?threshold=11
Send
Docs
Params
Authorization
Headers 7
Body
Scripts
Tests
Settings
Cookies
none
form-data
x-www-form-urlencoded
raw
binary
GraphQL
JSON
Schema
Beautify
1
Body
Cookies
Headers 5
Test Results
200 OK
29 ms
484 B
{} JSON
Preview
Visualize
[
{
"prodId": 6,
"name": "Laptop",
"price": 999.99,
"stockQuantity": 10
},
{
"prodId": 1,
"name": "Laptop",
"price": 999.99,
"stockQuantity": 4
},
{
"prodId": 2,
"name": "water",
"price": 999.99,
"stockQuantity": 10
},
{

7. Suggestions for Improvement

The following enhancements are recommended to further mature the system toward production readiness:

- Implement JWT-based authentication and role-based access control (ADMIN vs. CUSTOMER roles) to secure all API endpoints.
- Apply optimistic locking via the @Version annotation on the Product entity to prevent race conditions when concurrent orders compete for the same limited stock.
- Add pagination support to list endpoints (e.g., GET /api/products, GET /api/categories) to ensure scalable handling of large datasets.
- Introduce Data Transfer Object (DTO) classes to decouple API responses from internal entity structures, eliminating the circular reference issue and reducing over-exposure of internal fields.
- Integrate Spring Boot Actuator to expose health checks and operational metrics at /health and /metrics endpoints.
- Write unit tests using JUnit 5 and Mockito for the Service layer, with particular focus on edge cases in confirmOrder() and cancelOrder().
- Add Swagger/OpenAPI documentation to make the API self-documenting and explorable via an interactive interface.
- Implement soft-delete functionality for products and customers to preserve referential integrity with historical order records.

8. Lessons Learned

8.1 Design Before Coding

Working through the Entity-Relationship Diagram (ERD), relational schema mapping, and normalization up to 3NF prior to writing any code proved highly effective. The database structure remained stable throughout development without requiring major restructuring, and this approach was reflected in full marks for the System Design competency.

8.2 Business Logic Belongs in the Service Layer

Centralizing all business rules in the Service layer and maintaining thin controllers significantly simplified debugging and modification. When the cancel endpoint required corrections, only the service and controller required changes; the repository and entity classes remained unaffected.

8.3 Oracle Database Architecture

The project provided practical exposure to Oracle's Container Database (CDB) and Pluggable Database (PDB) architecture, which differs substantially from simpler databases such as MySQL or H2. Understanding that user accounts must be provisioned within a PDB rather than the root CDB is critical knowledge for enterprise Java development.

8.4 Importance of the @Transactional Annotation

The @Transactional annotation is essential to ensuring that the entire order confirmation process — encompassing stock validation, stock deduction, price locking, and total calculation — either succeeds in its entirety or is fully rolled back. Without this guarantee, a failure at any intermediate step could leave the database in an inconsistent state.

8.5 Value of Testing Business Rule Violations

The most valuable Postman tests were not the standard happy-path scenarios, but rather the violation tests — including attempting to confirm an oversold order (409 Conflict), cancelling a delivered order (409 Conflict), and verifying that stock is correctly restored after cancellation. These edge cases directly correspond to the evaluation criteria assessed under the Testing competency.

8.6 JSON Serialization in Bidirectional Relationships

Bidirectional JPA relationships require careful JSON serialization management from the outset. The @JsonManagedReference / @JsonBackReference pattern — or the use of DTOs — must be applied during entity design. Deferring this until after all entities have been built significantly increases the complexity of the fix.

9. Conclusion

This assignment constituted a comprehensive exercise in building a production-quality backend API from the ground up. The most instructive aspects were the deliberate design of the database schema prior to coding, the correct implementation of the stock-on-confirmation business rule, and the systematic validation of the system through Postman testing that included edge cases and business rule violations.

The project successfully delivers all required capabilities, including category and product management, customer management with multi-phone support, a complete order lifecycle (DRAFT → CONFIRMED → SHIPPED → DELIVERED / CANCELLED), atomic stock management with oversell prevention, order status history tracking, and low-stock reporting.

The experience reinforced the importance of disciplined software engineering practices — particularly upfront design, layered architecture, transactional integrity, and thorough testing — all of which are foundational to the development of reliable backend systems in enterprise environments.